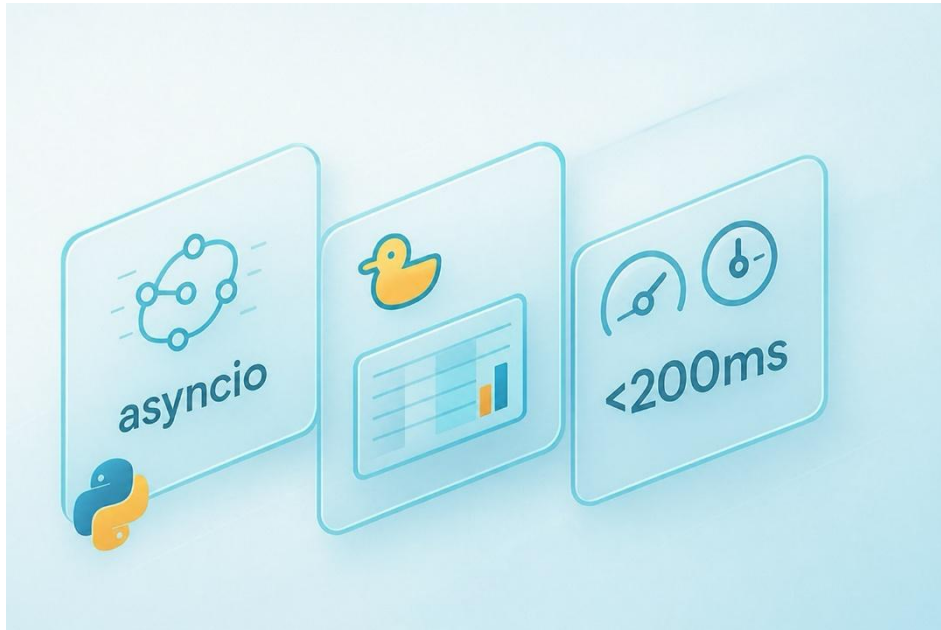


Carlos Cano Garrido

Álvaro Heredia Casado

TALLER 4/B – Procesado y Análisis de Datos con Python.



El objetivo principal de esta práctica es diseñar un sistema capaz de leer, procesar y visualizar en tiempo real datos provenientes de logs generados por una Unidad Central (CU) de una red 5G. Para ello, se realizará un script de Python aplicando principios de programación concurrente, gracias al uso de la librería *asyncio*. Además, se utilizarán herramientas como las expresiones regulares para filtrado y búsqueda de patrones en texto, Polars para el tratamiento eficiente de los datos o Dash para la visualización de estos en una página web.

La práctica consta de dos fases, que se expondrán en sus respectivas secciones en este documento.

- **Fase 1:** El sistema debe leer los logs de la red (Lectura a tiempo real, haciendo uso del script proporcionado por el profesor *log-sim.py* que vuelca logs de un archivo estático en otro, simulando la generación en tiempo real de estos), identificar los mensajes con información del volumen de tráfico DL a nivel SDAP y guardarlo en un objeto DataFrame. Se quiere una granularidad de 1 segundo en el tráfico agregado y distinción por UEs. Esta información por segundo debe visualizarse a tiempo real con Dash y ser guardada en un archivo Parquet cada 30 segundos.

- **Fase 2:** Añade una capa de complejidad al requerir que el sistema mantenga “memoria”. El objetivo es capturar los parámetros de configuración de red iniciales (PLMN, PCI y RNTI) por UE y añadirlos a la visualización en tiempo real del tráfico DL de la fase 1.

FASE 1

Para abordar la primera fase, se ha diseñado una arquitectura basada en el patrón Productor-Consumidor, implementada mediante la librería *asyncio* (tomando como referencia el esqueleto proporcionado por el profesor en el script *do-dash.py*). Este diseño permite que la velocidad de lectura de los logs no condicione la de procesado y escritura de estos, las líneas leídas se almacenan temporalmente en una cola, evitando que se pierdan logs si el proceso de escritura tardase más.

Por otro lado, la herramienta de visualización (Dash) funciona de forma síncrona y bloqueante. Para evitar que el servidor web congele la ejecución y detenga la lectura de la red, la ejecución de Dash se aísla en un hilo del sistema operativo independiente.

Arquitectura Concurrente: Productor y Consumidor

El sistema se divide en dos rutinas principales que se ejecutan de forma cooperativa compartiendo los recursos de un mismo hilo, de manera que cuando una tarea está inactiva, la otra toma el control para seguir procesando información sin detener el programa.

- **El Productor (*tail_file_producer*):** Actúa como un recolector. Su función es mantener el archivo *cu-lan-ho.log* abierto, leyendo continuamente las nuevas líneas conforme el simulador (*log-sim.py*) las escribe. Filtra la información, la empaqueta y la introduce en la cola.
- **El Consumidor (*consumer*):** Extrae los paquetes de datos de la cola a su propio ritmo, los transforma en *DataFrame*, los agrega al *DataFrame* global almacenado en memoria y, cada 30 segundos, lo guarda en el archivo *parquet*.

```

async def main():
    print(" ")
    print(f"Procesador 5G (Fase 1) :: v{version}")
    print(" ")

    asyncio.create_task(tail_file_producer(data_file, queue))
    asyncio.create_task(consumer(queue))

    print("[*] Iniciando servidor web de Dash en http://127.0.0.1:8050")
    await asyncio.to_thread(
        app.run,
        host="127.0.0.1",
        port=8050,
        debug=False,
    )

```

`asyncio.create_task` pone a ejecutarse ambas funciones de forma concurrente en el mismo hilo mientras que con `asyncio.to_thread` se delega la ejecución de Dash a otro hilo del SO, evitando el bloqueo.

Extracción de Datos mediante Expresiones Regulares

Los *logs* de red se generan en texto plano no estructurado. De entre todos los logs generados, se quiere extraer el volumen de bytes en enlace descendente en la capa SDAP por usuario. Esta información tiene el siguiente formato en los logs:

```

2026-01-19T08:55:47.070895 [SDAP ] [D] ue=0 psi=2 QFI=1 DRB2 DL: TX PDU.
QFI=1 pdu_len=76

```

Se quiere extraer el timestamp, el ue y la longitud del paquete de datos (`pdu_len`). Para extraer eficientemente la información útil ignorando el "ruido", se ha diseñado la siguiente expresión regular (Regex):

```

regex_sdap = re.compile(r'^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})\.\d+.*?\[SDAP\s*\].*?ue=(\d+).*?DL: TX PDU.*?pdu_len=(\d+)', re.IGNORECASE)

```

Este patrón permite aislar los mensajes específicos de la capa SDAP de tráfico DL, dividiendo la información en tres grupos:

- Grupo 1: La marca de tiempo (*Timestamp*) del evento.
- Grupo 2: El identificador del usuario (`ue`).
- Grupo 3: El volumen de datos transmitido en bytes (`pdu_len`).

Productor: Lectura de datos y agregación temporal

La función asíncrona lee las líneas que *log-sim.py* va escribiendo en el archivo "cu-lan-ho.log". Cada iteración del bucle *while* lee una línea del archivo usando *readline()*.

Haciendo uso de la expresión regular se extraen los datos de interés de la capa SDAP. Esta lectura tiene una granularidad de milisegundos, pero la especificación de la práctica pide agregación del volumen de bytes por segundo, para conseguir

esta agregación por segundos, se hace uso de funciones de la librería *datetime*, concretamente la línea:

```
log_segundo = log_dt.replace(microsecond=0)
```

Aplana el *datetime* de cada log a los segundos, de modo que, todos los logs que entren dentro del mismo segundo, aunque sean milisegundos distintos, se procesarán de forma conjunta, y se agregarán los bytes en DL por UE y por segundo. Un diccionario auxiliar llamado *buffer_segundo* acumula los bytes por UE y se limpia cada vez que ha cambio de segundo, justo después de ser enviado (mediante *await data_queue.put()*) a la cola que recibirá el consumidor.

```
async def tail_file_producer(path: str, data_queue: asyncio.Queue):
    file = Path(path)
    print(f"[*] Esperando la llegada de logs del CU: {data_file}")

    while not file.exists():
        await asyncio.sleep(0.5)

    with open(path, "r", encoding="utf-8") as f:
        print(f"[+] Logs detectados. Leyendo en tiempo real...")
        f.seek(0, 2)

    buffer_segundo = {}
    segundo_actual_log = None

    while True:
        line = f.readline()

        if not line:
            await asyncio.sleep(0.05)
            continue

        match = regex_sdap.search(line)
        if match:
            log_dt_str = match.group(1)
            ue_id = match.group(2)
            dl_bytes = float(match.group(3))

            log_dt = datetime.fromisoformat(log_dt_str)

            log_segundo = log_dt.replace(microsecond=0)

            if segundo_actual_log is None:
                segundo_actual_log = log_segundo

            if log_segundo > segundo_actual_log:
                if buffer_segundo:
                    await data_queue.put((segundo_actual_log, buffer_segundo.copy()))
                    buffer_segundo.clear()

                segundo_actual_log = log_segundo

            buffer_segundo[ue_id] = buffer_segundo.get(ue_id, 0.0) + dl_bytes
```

Consumidor: Transformación y Almacenamiento de Datos

La función asíncrona *consumer* actúa como el receptor de la información preprocesada. A través de la instrucción *await data_queue.get()*, extrae los paquetes de la cola en cuanto el Productor los genera. Cada paquete extraído

contiene el segundo exacto del *log* y el diccionario con los volúmenes de bytes asociados a cada UE.

A continuación, el diccionario se transforma en un *DataFrame* temporal mediante la librería Polars, elegida por su alta eficiencia en el manejo de matrices de datos. Este bloque temporal se concatena verticalmente al *DataFrame* global *agg_df*:

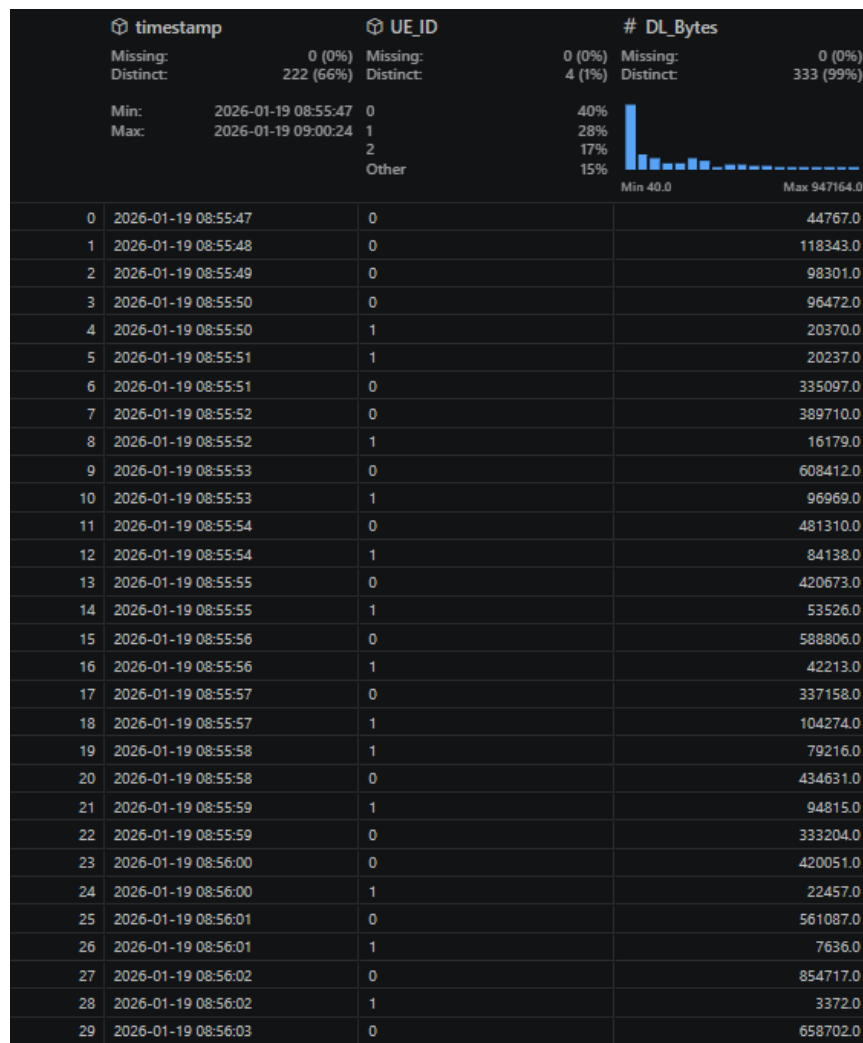
```
agg_df = pl.DataFrame(  
    schema={  
        "timestamp": pl.Datetime,  
        "UE_ID": pl.Utf8,  
        "DL_Bytes": pl.Float64  
    }  
)
```

el cual reside en la memoria RAM y servirá como fuente de lectura para la gráfica de Dash. Cuando cada paquete leído de la cola se ha procesado, el consumidor lo indica con la instrucción *data_queue.task_done()*, esto mantiene al contador interno de la cola de *asyncio* sincronizado, asegurando que todas las tareas pendientes en la cola se realicen antes de terminar la ejecución.

A su vez, el Consumidor gestiona el almacenamiento de los datos fuera de memoria RAM. Cada 30 segundos, guarda el *DataFrame* global en un archivo *parquet*. A diferencia del Productor (que se rige por el tiempo del *log*), aquí se hace uso del reloj interno de la máquina (*time.time()*) para medir el tiempo real de ejecución.

```
async def consumer(data_queue: asyncio.Queue):  
    global agg_df  
    ultimo_guardado_parquet = time.time()  
  
    while True:  
        dt_timestamp, ue_data = await data_queue.get()  
  
        nuevas_filas = [  
            {"timestamp": dt_timestamp, "UE_ID": str(ue), "DL_Bytes": vol}  
            for ue, vol in ue_data.items()  
        ]  
  
        if nuevas_filas:  
            df_temporal = pl.DataFrame(nuevas_filas, schema=agg_df.schema)  
            agg_df = pl.concat([agg_df, df_temporal], how="vertical")  
            print(f"► Procesado 1s (Log Time: {dt_timestamp.strftime('%H:%M:%S')}): {len(nuevas_filas)} UEs.")  
  
            tiempo_actual = time.time()  
            if tiempo_actual - ultimo_guardado_parquet >= 30.0:  
                if len(agg_df) > 0:  
                    print(f"\nHan pasado 30 segundos: Guardando {len(agg_df)} registros en el Parquet...\n")  
                    agg_df.write_parquet(parquet_file)  
                    ultimo_guardado_parquet = tiempo_actual  
  
            data_queue.task_done()
```

** Haciendo uso de la extensión de Visual Studio Code: “Data Wrangler” se visualiza el .parquet generado por la ejecución:



Visualización con Dash

Para la representación gráfica, se crea un servidor web utilizando Dash. Como se mencionó anteriormente, para evitar que el servidor HTTP bloquee el bucle asíncrono de procesamiento de *logs*, la ejecución de la aplicación Dash se ha delegado a un hilo independiente del sistema operativo mediante la instrucción *asyncio.to_thread*.

La actualización de la gráfica, sin necesidad de que el usuario recargue la página, se logra mediante la combinación de un componente *dcc.Interval* y un decorador *@app.callback*. El componente de intervalo actúa como un reloj interno configurado a 1000 milisegundos (1 segundo), el cual dispara automáticamente la función de actualización de la gráfica (*update_graph*).

Dentro de esta función de *callback*, el sistema lee el *DataFrame* global (*agg_df*) que el Consumidor mantiene actualizado en la memoria RAM. Para optimizar el trazado de la gráfica, la estructura de Polars se convierte temporalmente a Pandas. Finalmente, el código itera sobre los identificadores de los UEs detectados, añadiendo una traza (*trace*) independiente para cada uno, lo que permite visualizar el volumen de tráfico individual de cada UE a lo largo del tiempo.

```
app = Dash(__name__)

app.layout = html.Div([
    html.H2("Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)", style={'font-family': 'Arial'}),
    dcc.Graph(id="live-graph"),

    dcc.Interval(id="interval", interval=1000, n_intervals=0),
])

@app.callback(
    Output("live-graph", "figure"),
    Input("interval", "n_intervals"),
)
def update_graph(n):
    fig = go.Figure()

    if len(agg_df) == 0:
        return fig

    pdf = agg_df.to_pandas()

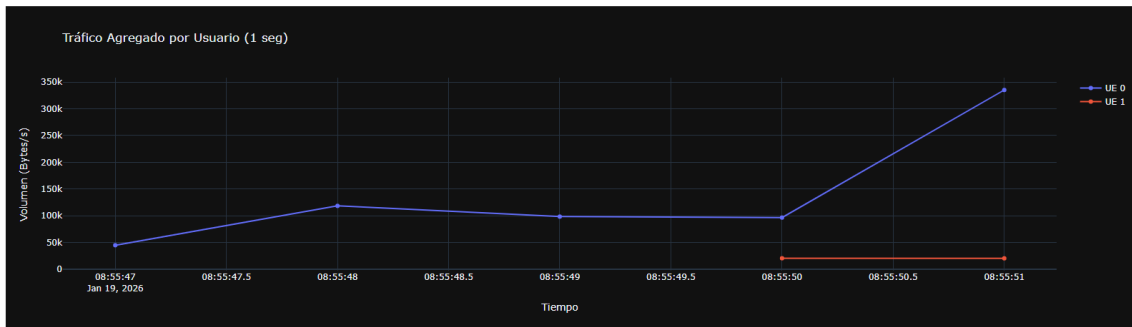
    for ue in pdf['UE_ID'].unique():
        datos_ue = pdf[pdf['UE_ID'] == ue]
        fig.add_trace(
            go.Scatter(
                x=datos_ue['timestamp'],
                y=datos_ue['DL_Bytes'],
                mode="lines+markers",
                name=f"UE {ue}"
            )
        )

    fig.update_layout(
        xaxis_title="Tiempo",
        yaxis_title="Volumen (Bytes/s)",
        title="Tráfico Agregado por Usuario (1 seg)",
        template="plotly_dark",
        hovermode="x unified"
    )

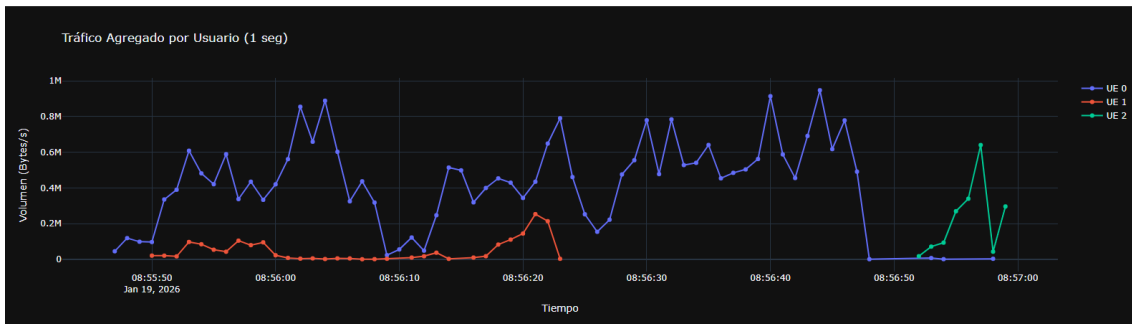
    return fig
```

El resultado tras ejecutar simultáneamente *log-sim.py* y *log-parser-fase1.py* es la siguiente gráfica corriendo en <http://127.0.0.1:8050/> con el volumen de bytes en DL por UE en la capa SDAP con actualización a tiempo real.

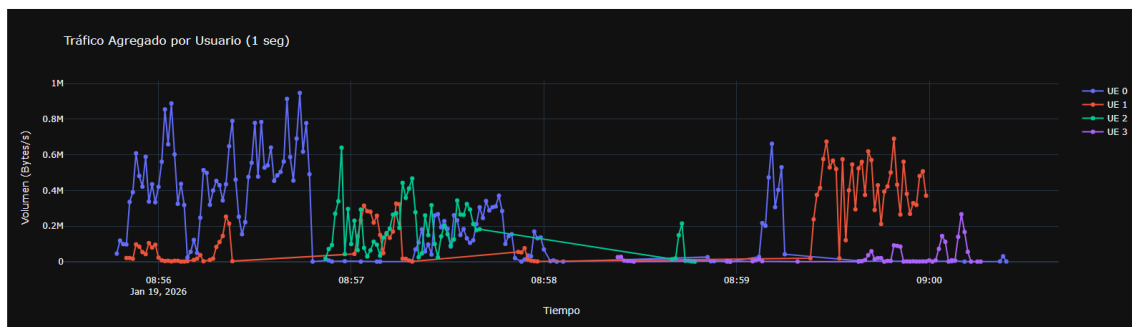
Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)



Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)



Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)



FASE 2

La segunda fase añade una capa de complejidad: el sistema pasa de un procesamiento "sin estado" (donde cada línea de log contiene toda la información necesaria) a un procesamiento "con estado". El objetivo es capturar los parámetros de configuración de red iniciales (PLMN, PCI y RNTI) asignados a cada UE y añadirlos a la visualización.

El reto principal radica en que esta información de contexto se emite en mensajes al inicio de la conexión una única vez, mientras que el volumen de datos se emite continuamente en líneas separadas más adelante dentro del archivo de logs. Por tanto, el sistema necesita recordar quién es cada usuario.

Extracción del Contexto de Red (Nueva Regex)

Para extraer la información de control, se ha añadido una segunda expresión regular orientada a la capa CU-UEMNG, que contiene la información deseada por UE conectado.

```
regex_ue_setup = re.compile(r'\[CU-UEMNG\s*\].*?ue=(\d+).*?plmn=(\d+).*?pci=(\d+).*?rnti=(0x[0-9a-fA-F]+)', re.IGNORECASE)
```

Esta expresión captura en cuatro grupos el identificador del usuario (ue) y sus parámetros de red asociados (plmn, pci y rnti).

Productor: Gestión de Memoria Dinámica

Para dotar al sistema de estado, se implementa una estructura de datos en el Productor denominada memoria_ue. Este diccionario servirá para almacenar el contexto de red.

El flujo de lectura en el bucle while se modifica para evaluar dos condiciones por cada línea leída:

1. Captura de parámetros de red: Si la línea coincide con la expresión de [CU-UEMNG], el programa guarda el PLMN, PCI y RNTI en el diccionario memoria_ue los valores para cada UE, y salta a la siguiente línea.
2. Agregación de tráfico: Si la línea coincide con la capa [SDAP], se acumulan los bytes en el buffer_segundo, igual que en la Fase 1.

La lógica del productor es la misma que en la primera fase, añadiendo la captura de la nueva expresión regular:

```
match_setup = regex_ue_setup.search(line)
if match_setup:
    ue_id = match_setup.group(1)
    memoria_ue[ue_id] = {
        "PLMN": match_setup.group(2),
        "PCI": match_setup.group(3),
        "RNTI": match_setup.group(4)
    }
    continue
```

y la agregación de esos parámetros al diccionario que se manda cada segundo a la cola.

```

paquete_final = {}
for ue, bytes_totales in buffer_segundo.items():
    info_red = memoria_ue.get(ue, {"PLMN": "N/A", "PCI": "N/A", "RNTI": "N/A"})
    paquete_final[ue] = {
        "DL_Bytes": bytes_totales,
        "PLMN": info_red["PLMN"],
        "PCI": info_red["PCI"],
        "RNTI": info_red["RNTI"]
    }

await data_queue.put((segundo_actual_log, paquete_final))
buffer_segundo.clear()

```

Consumidor y Visualización con Dash

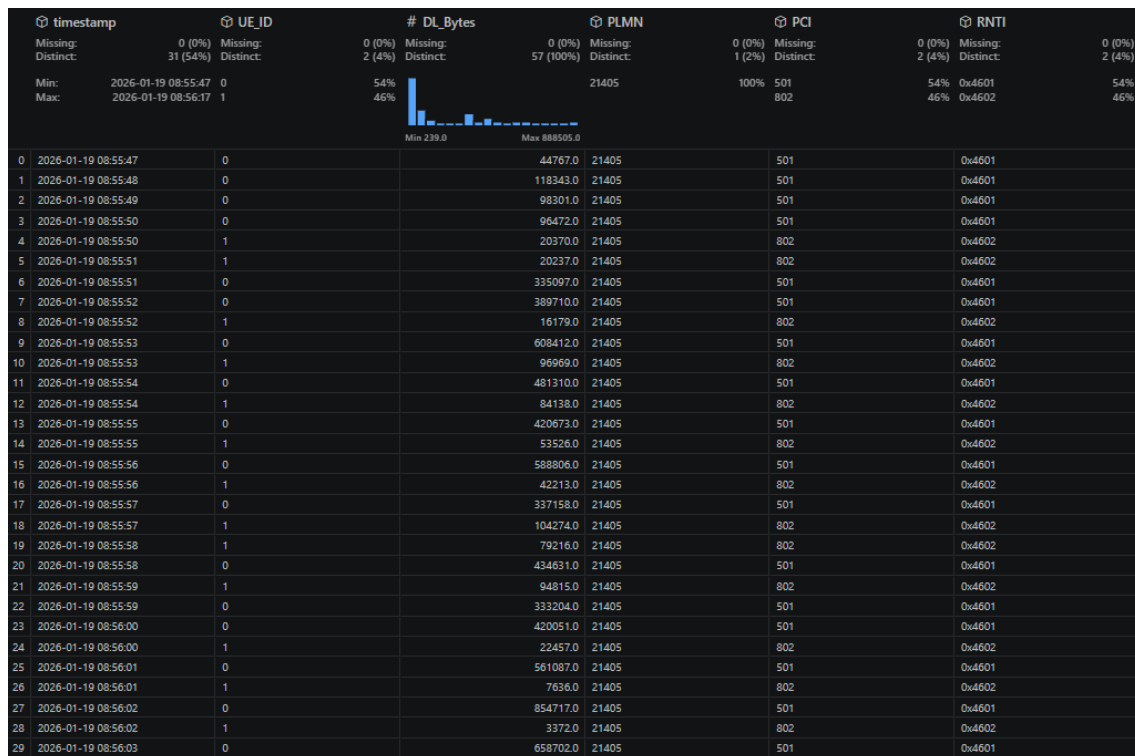
Para asimilar estos nuevos datos, el esquema del *DataFrame* global de Polars (*agg_df*) ha sido ampliado, definiendo el tipo de dato (*pl.Utf8*) para las nuevas columnas de PLMN, PCI y RNTI.

```

agg_df = pl.DataFrame(
    schema={
        "timestamp": pl.Datetime,
        "UE_ID": pl.Utf8,
        "DL_Bytes": pl.Float64,
        "PLMN": pl.Utf8,
        "PCI": pl.Utf8,
        "RNTI": pl.Utf8
    }
)

```

El Consumidor extrae el paquete de la cola, transforma las filas y las concatena al histórico global, manteniendo el mismo guardado en Parquet cada 30 segundos, que tiene la siguiente estructura.



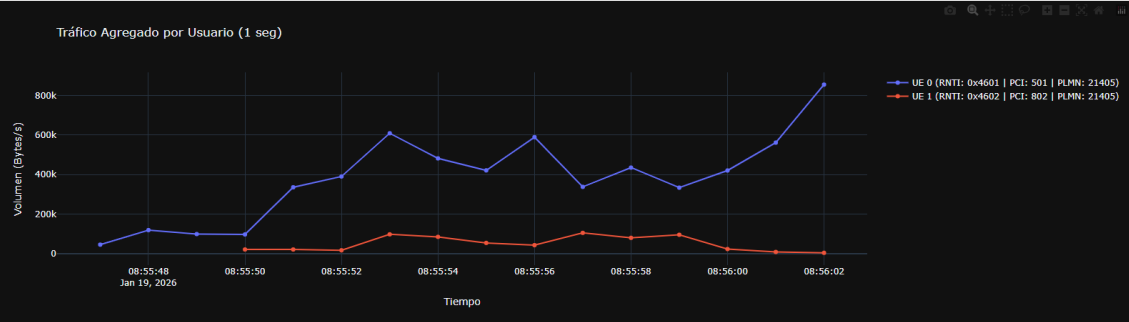
Finalmente, la función asíncrona de Dash (`update_graph`) aprovecha esta nueva información. Al iterar sobre los UEs únicos para dibujar sus trazas en la gráfica, el código extrae los parámetros de red de la última fila disponible para ese usuario y los inserta dinámicamente en la etiqueta de la leyenda (`name`).

```
ultimo_dato = datos_ue.iloc[-1]
etiqueta_leyenda = f"UE {ue} (RNTI: {ultimo_dato['RNTI']} | PCI: {ultimo_dato['PCI']} | PLMN: {ultimo_dato['PLMN']})"

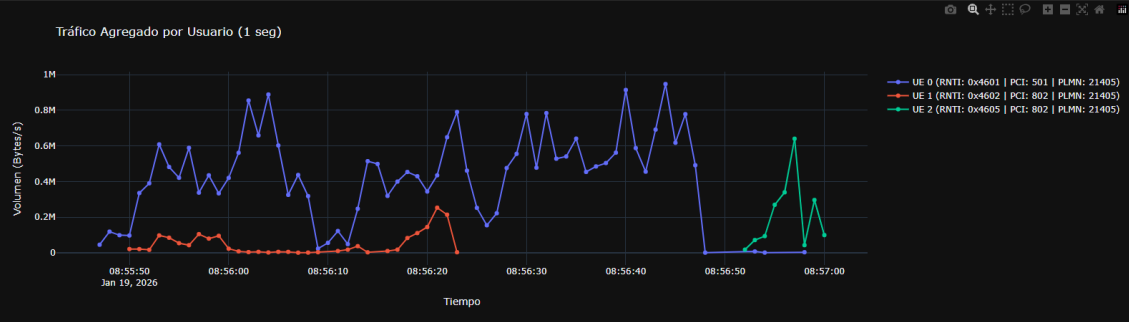
fig.add_trace(
    go.Scatter(
        x=datos_ue['timestamp'],
        y=datos_ue['DL_Bytes'],
        mode="lines+markers",
        name=etiqueta_leyenda
    )
)
```

El resultado final, nos permite visualizar los datos de tráfico DL por usuario, con información específica de cada uno de ellos a tiempo real.

Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)



Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)



Telemetría 5G: Volumen SDAP en Downlink (Tiempo Real)

